

Document number: P2182R1
Date: 2020-11-16
Audience: SG21
Reply-to: Andrzej Krzemiński <akrzemi1 at gmail dot com>
Joshua Berne <jberne4 at bloomberg dot net>
Ryan McDougall

Contract Support: Defining the Minimum Viable Feature Set

We observe that "C++20 contracts" consisted of two parts. One part was clear, well understood, widely agreed upon, and useful to various groups of C++ users. The other part rose a lot of controversies for various reasons and ultimately led to the removal of the entire feature.

In paper [\[P2114+0\]](#) we have listed a subset of use cases we believe represent useful improvements to the language, offer an incalculable positive value for most users, and reflect the uncontroversial part of "C++20 contracts". In this paper we provide the rationale for our choice. We wish to show that our selection:

- is big enough to be called a Minimum Viable Product (MVP) and get support of the majority of C++ programmers;
- is small enough that we can make it ready in a relatively short period of time;
- does not require controversial design, or features not already implemented in other languages;
- does not prevent the addition of any of the remaining features of "C++20 contracts", or other discussed alternatives, in subsequent stages.

0. Revisions

0.1. R0 → R1

- Added section Design Objectives and Programming Model.
- Provided some examples of what is proposed for the MVP.
- Augmented the discussion of C++20 Contracts features not in MVP.
- Added section Other Use-Cases Compatible with Our Model.
- Added section Use-Cases Incompatible with Our Model.

1. Design Objectives and Programming Model

1.1. Design Objectives

Our primary objective is to enable programmers to **improve safety** of their programs, by the following means:

- detecting bugs,
- enabling predictable behavior in the face of such bugs.

Secondarily, we aim at leaving the door open for other objectives in the future, such as supporting contract-based code transformations, as long as:

- they fit into our programming model,
- they do not compromise our ability to satisfy the primary objective.

1.2. The Programming Model

1.2.1. Declaring criteria for bugs

The contract support facility that we propose enables programmers to declare in the source code that certain values of objects observed at certain times necessarily indicate a bug somewhere in the program.

```
T& Container::get(int i)
[[pre: i >= 0]]           // precondition
[[pre: i < size()]]       // precondition
{
    [[assert: buffer_ != nullptr]]; // block-scope invariant
    return buffer_[i];
}
```

Here, the two preconditions specify that whenever function `Container::get` is invoked, just before its execution, if either of expressions `i >= 0` or `i < size()` were to evaluate to `false`, it would indicate a bug in the program. Class author can make that declaration because she understands the human-to-human class/function contract: that the `int` value passed to this function represents an abstract notion of an *index* in a container.

The assertion inside the function body indicates that when control reaches this statement, if expression `buffer_ != nullptr` were to evaluate to `false`, it would indicate a bug in the program. The class author can make that declaration, because she knows the purpose for which she put pointer `buffer_` as a class member, and she knows the rules based on which it will change value.

In either case, the programmer has a knowledge about the program that the machines do not. Through these declarations machines obtain part of this knowledge: they can now precisely determine that a program has a bug.

Different machines can make different use of these contract annotations. For instance, static analyzers can perform a symbolic evaluation of the program and determine if there exist combinations of external input that lead to any of the contract annotations producing value `false`. Compilers, under special flags, can inject runtime-checks in place of these declarations, which would make the above function behave as if it was defined so:

```
T& Container::get(int i)
{
    if ((i >= 0) == false) { std::cerr << violation("i >= 0"); std::abort(); }
    if ((i < size()) == false) { std::cerr << violation("i < size()"); std::abort(); }
    if ((buffer_ != nullptr) == false) { std::cerr << violation("buffer_ != nullptr"); std::abort(); }
    return buffer_[i];
}
```

Tools can do these things only because the programmer gave them this additional information about what constitutes a bug.

1.2.2. Controlling the tools

Tools are resource-limited. In order for them to make an effective use of contract annotations, the programmer, or someone else, needs to be able to instruct the tools to treat different contract annotations differently. Usually, it would mean indicating a subset of contract annotations as more important, taking some action on them, and ignoring the rest.

In case of a static analyzer, a programmer may want to indicate a subset *A* of all contract annotations, and instruct the tool to statically check if any contract annotation from *A* can be potentially violated assuming that all other contract annotations are never violated.

Analogously, another way of detecting bugs is for the compiler to inject runtime-checks into compiled program based on contract annotations. Whenever a runtime check detects contract violation, it logs this fact (in this scenario it is irrelevant whether the program stops or continues after a logged failure). This way the compiled program does two things: (1) its documented business logic that the customers expect, and (2) detecting bugs behind the scenes. However, performing additional runtime checks may make the program perform inacceptably slow. Therefore, the programmer may want to indicate a subset *B* of all contract annotations, and instruct the compiler to only inject runtime checks based on contract annotations from *B*. This is a compromise between detecting bugs in a running program and maintaining satisfactory program performance.

In either of these cases, a programmer needs to have a way to designate a subset of contract annotations. Many ways of designating such subset have been explored:

- flagging individual contract annotations as belonging to the subset using structured comments,
- associating the subset with a given class, namespace, translation unit, module, or other component in the program,
- assigning tags to contract annotations (through structured comments) and then building the subset based on those tags,
- indicating that only preconditions (as opposed to postconditions, assertions and invariants) belong to the subset,
- defining the subset in a special configuration file based on the above criteria,
- defining the subset in using the compiler's switches.

There may be more reasons for designating the subset, other than saying which contract annotations to ignore. For one example, in the case of compiler injecting runtime checks based on contract annotations, a programmer might want to indicate after which failed runtime checks program should definitely stop, and after which it had better continue.

All the above remains compatible with our programming model. But we do not propose it as part of MVP. The only form of the additional tool control that we actually propose for MVP is the ability, from outside the source code (that is, from compiler command line, or similar), to require that runtime checks are injected from all contract annotations in the program, and similarly to require that no runtime check is injected from any contract annotations. This modes could be called "enable all" and "disable all".

There are two reasons for enabling this minimum control. One is to cover the use case that is common, well understood and already available in other languages. Second is to alert the users that when they put predicates in contract annotations, the two situations need to be accounted for:

1. The predicate may not be runtime-evaluated at its corresponding location. Therefore, any code that relies on the predicate's side effects will be broken.
2. The predicate may be runtime evaluated. Therefore, any side effect that the evaluation of the predicate will trigger may break the program.

1.2.3. Other applications of the model

Technically, it is possible to use contract annotations for purposes different than bug detection. One such notable purpose is code transformations that guarantee to leave the bug-free programs unchanged and possibly optimized, while at the same time arbitrarily changing the semantics of programs that have bugs diagnosable through contract annotations. This is sometimes (imprecisely) referred to as contract-based optimizations.

Contract-based optimizations are compatible with our programming model (users declare what they consider a bug, and tools make use of these declarations); but concerns have been expressed about the potential interference with our primary goal: improving safety. Every useful program in practice does contain bugs, and often programs can behave reasonably well even in the face of these bugs. Allowing contract-based optimizations may turn programs that behave reasonably in the face of bugs into programs that behave unreasonably in the face of bugs. Wherever it can be demonstrated that any secondary goals are in a direct and an irreconcilable conflict with our primary goal, we commit to resolving any such conflict in favor of our primary goal: improving safety.

2. Analysis

In this document we use the term *C++20 Contracts* to refer to contract support facilities in C++ Standard draft just before the removal — [\[N4820\]](#) (which is effectively equivalent to [\[P0542R5\]](#)) — as well as pieces of the associated proposals seen by EWG that had support in polling, such as:

- [\[P1290R3\]](#)
- [\[P1344R1\]](#)
- [\[P1429R3\]](#)
- [\[P1607R1\]](#)
- [\[P1711R0\]](#)

[\[P2076r0\]](#) lists the points of disagreement about the previous incarnation of contracts design:

- Continuation Mode
- Build Levels
- Global Toggles (also known as: external remapping controls)
- Literal Semantics (also known as: in-source controls)
- Assumptions

We observe that all these features fall into category "controlling the tools" in our model. We claim that if we eliminated the use cases for fine-grained tool control, the remainder would still satisfy a huge, consistent and well understood demand. Addressing it would be a valuable and usable feature that would offer a great help in enforcing program correctness. The value in preventing bugs and enforcing program correctness comes from being able to:

1. Just express preconditions, postconditions, and assertions as C++ predicates. This enables a portable language for communicating what constitutes a bug. This information can be consumed by many different tools as well as human programmers.
2. Transform them into runtime-checks at an all-or-none granularity, leaving open the design of finer control to future work. This enables the guarantee that program will never continue after detecting a bug defined by the contract annotations.

In the reminder of the paper we elaborate on the features of MVP and show how they are motivated by the use cases nominated in [\[P2114r0\]](#). We also discuss how the support for the remaining use cases can be added atop of MVP in subsequent phases.

2.1. Features in MVP

We selected the minimal use cases to allow the widest benefit for the most general section of users, without resorting to known points of contention. We had in mind a simple feature that consisted of the C++20 Contract syntax, a single build option to enable or disable checks, and a (conditionally) user-definable violation handler that is invoked when runtime-enabled checks are violated.

2.1.1. Contract declarations

The most important thing is the ability to express the predicates in function declarations as compiler-checked C++ expressions. This is uncontroversial. This enables:

- compiler to report type-system errors from incorrectly spelled constraints
- generation of run-time code — such as contract checks, or unit tests
- annotation aware static analysis

The contract annotations look like this:

```
int Accumulator::running_max(int i)
[[pre: i >= 0]]      // precondition
[[post r: r >= 0]]   // postcondition
{
    [[assert: stored_ >= 0]]; // block-scope invariant
    return stored_ = std::max(i, stored_);
}
```

Compared to C++20 contracts, there is no room for contract-levels: `default`, `audit` or `axiom`. Also no room for "literal semantics", such as `ignore`. We only have a predicate and, in the case of postconditions, the name of return value.

In fact, this feature alone would already satisfy many users.

Use cases covered:

- [dev.reason.knowl](#)
- [dev.readable.syntax](#)
- [dev.readable.keywords](#)
- [dev.parsable](#)
- [dev.tooling](#)
- [cppdev.syntax.familiar](#)
- [cppdev.syntax.cpp](#)
- [cppdev.syntax.reuse](#)
- [cppdev.location](#)
- [api.communicate.inputsoutputs](#)
- [api.establish.preconditions](#)
- [api.establish.postconditions](#)

- [api.express.values](#)
- [api.contract.errorhandling](#)
- [arch.nomacros](#)
- [arch.complete](#)
- [sdev.bestpractices](#)
- [jdev.understand.contracts](#)
- [jdev.understand.keywords](#)
- [adev.fast](#)
- [teach.bestpractices](#)
- [teach.standardized](#)
- [teach.portable](#)

2.1.2. Turning on and off all runtime checks

We believe it is essential that we meet two somewhat contradictory requirements:

1. A guarantee that some action is taken whenever a contract is violated
2. A guarantee that there is no runtime overhead for encoded contracts

The former is a correctness requirement, and the latter is performance requirement (after assurance — further discussion about assurance can be found in [\[P1517R0\]](#)).

This suggests that there needs to be at least some control, at build time, over what effect contract annotations have on the final executable. For the MVP we suggest a single flag which turns checking "on" or "off", where:

- "Off" requires that predicates in contract annotations are syntax-checked but otherwise do not affect the resulting executable.
- "On" requires that runtime checks are inserted in place of contract annotations. When any such predicate check returns `false`, violation handler is invoked.

This is a subset of what [\[P0542R5\]](#) offered, and should come with similar caveats that that proposal had for build modes — where mixing modes is only conditionally supported.

Finer-grained control and additional semantics to failed checks would be left to extensions and future proposals.

Use cases covered:

- [dev.reason.sideeffects](#)
- [int.conform.postconditions](#)
- [int.control.build](#)
- [int.runtime.unchecked](#)
- [qdev.checkall](#)

2.1.3. Reporting contract violation

On platforms where it makes sense, each contract violation detected at run-time should be reported to the users — including what failed and where in the source code.

Use cases covered:

- [api.establish.check](#)
- [api.establish.values](#)
- [int.conform.violation](#)
- [int.violations.information](#)
- [jdev.understand.violations](#)
- [jdev.understand.aborting](#)
- [jdev.understand.buildviolation](#)

2.1.4. Customizing how contract violations are reported

The semantics and implementation of contract violation handling has significant impact on the extensibility and usability of the feature. However, we cannot offer a fully usable solution without some additional annotations put in individual contract annotations, and this is not proposed for the MVP. What we propose instead is that the Standard only requires that a violation handler is called, but:

- We do not require that the user must be able to customize the handler.
- We do not require that the handler actually logs anything anywhere.
- We do not require a uniform behavior for different contract annotations.
- We do not require that `std::abort()` or `std::terminate()` is called at the end of the handler.

However, an implementation that would offer all the above would be conforming.

Use cases covered (implementation defined):

- [int.violations.common](#)

2.2. C++20 Contracts Features not in MVP

This section lists use cases that are essential for a contract programming facility, were present in some form in C++20 Contracts, but we decided to leave them out for the MVP.

2.2.1. Continuation after failed run-time check

The ability to control whether a program is halted after a violation handler is called: whether for all runtime-checked contract annotations, or for a subset of them. This would address [sdev.maturity](#) and [large.newcompiler](#).

This ability can be introduced on top of MVP as secondary annotations on individual contract annotations (as described in [\[P1332R0\]](#)), and additionally requiring more build modes.

2.2.2. Controlling run-time checks based on cost

The ability to declare the relative cost of runtime-evaluating the predicate inside the contract annotation against the runtime cost of evaluating the function that the contract annotation appertains to would address use cases:

- [crit.more.coverage](#),
- [dev.reason.cost](#),
- [crit.production.checking](#).

Contract levels, such as `default` and `audit`, tried to address this, but the design of this feature had flaws:

- The proposed syntax cannot handle templates where the level may depend on template arguments.
- Two levels (or even three) cannot be enough.
- Levels is not enough to convey essential meta information.
- Syntax is not extensible.
- Syntax is uncomfortable.

This functionality can be introduced on top of MVP as annotations indicating how expensive a contract statement is relative to the body of the function.

2.2.3. Injected Facts (Assumptions)

This was highly controversial. An extended analysis can be found in [\[P2064r0\]](#). Use cases covered: [pdev.footgun](#) and [hardware.performance](#).

The best explored use case for this is when there is a single fact injected into a program that is known to produce a huge performance gain; e.g., that the size of some buffer will always be a multiple of 8. Such injected fact can change the performance characteristics of a program from unacceptably slow to really fast. In that case the risk of corrupting the program is justified. This use case can be handled by a separate feature, such as [\[P1774r3\]](#). Alternatively, users can stick to vendor-specific extensions, such as `__builtin_assume()`.

The other use case is when all, or a group, of contract annotations is turned simultaneously into multiple injected facts, without previous knowledge if and to what extent this might improve program performance. Turning literally all contract annotations into injected facts is possible if a programmer installs a custom violation handler containing `__builtin_unreachable()`.

The remaining use case for turning a selected group of contract annotations into injected facts could in principle be achieved in the future by introducing another second-level annotation in contract annotations.

2.2.4. Indicating inexpressible predicates

This would cover use cases [qdev.tooling.control](#) and [api.express.unimplementable](#). Controversies arose because of mixing inexpressibility with levels, and taking word "axiom" to mean `__builtin_assume()`.

Can be implemented atop of MVP either as meta-information on contract declarations, or by annotating functions as inexpressible as described in [\[P2176r0\]](#).

2.3. Other Use-Cases Compatible with Our Model

This section lists use cases not addressed in the MVP that are compatible with our programming model that could be added in the future.

2.3.1. Grouping the annotations

The ability to organize contract annotations into groups would address a lot of use cases. This could be achieved by assigning tags to individual contract annotations. Later, upon assembling the entire program, the person in charge could instruct the tool (e.g., compiler) to treat contract annotations with a given tag differently than contract annotations with different tags. This way desired semantics could be assigned to groups of contract annotations (rather than to individual annotations or to all annotations).

This would cover use cases:

- [large.complex](#),
- [int.build.control](#),
- [int.build.control2](#),
- [int.conrol.subsets.build](#),
- [int.control.subsets](#),
- [lib.maintenance.nowhining](#),
- [lib.integration.nowhining](#),
- [sdev.maturity](#),
- [sdev.control](#),
- [qdev.tooling](#),
- [qdev.tooling.control](#),
- [qdev.tooling.undefined](#),
- [qdev.tooling.undefinedkinds](#),
- [crit.control](#),
- [crit.locality](#),
- [crit.more.coverage](#),
- [large.complex](#),
- [large.observation](#),
- [large.introduction](#),
- [large.separability](#),
- [large.newenvironment](#),
- [large.newcompiler](#),
- [large.nogoingback](#),
- [large.simulation.disable](#),
- [large.simulation.enable](#),
- [large.simulation.ignore](#),
- [large.perfcontrol.build](#),
- [large.perfcontrol.runtime](#),
- [large.narrowing](#),
- [embedded.minimize](#).

2.3.2. More contract annotations

There are more contract annotations that we could add, which would give even more information to tools looking for bugs.

One, class invariants:

```
class TimeOfDay
{
    int minutes_; // since midnight
    [[assert: minutes_ > 0]]; // class invariant
    [[assert: minutes_ < 1440]]; // class invariant

public:
    explicit TimeOfDay(int mins)
        [[pre: mins >= 0]]
        [[pre: mins < 1440]];

    int minutes() const
        [[post r: r >= 0]]
        [[post r: r < 1440]];
};
```

This would cover use cases:

- [api.establish.validate_invariants](#),
- [api.establish.invariants](#),
- [cppapi.invariants](#),
- [api.class.publicinterface](#),
- [api.class.publicinvariants](#),
- [api.class.publiccalls](#),
- [api.class.baseinterface](#),
- [api.class.baseinvariants](#),
- [api.class.basecalls](#),
- [api.class.privateinterface](#),
- [api.class.privateinvariants](#),
- [api.class.privatecalls](#).

2.3.3. Referring to old values of modified objects

The ability in function postconditions to refer to values of objects observed at the point the function was entered. For instance, `.push_back()` postcondition could state that the value of `.size()` upon exit from function should be greater y one than the value of `.size()` upon entering the function.

There is a number of ways the syntax for contract annotations can be extended in the future to accomodate this. Here we list two possibilities:

```
void container::push_back(T const& v)
    [[post: size() == oldof(size()) + 1]];

void container::push_back(T const& v)
    [[post old_size = size(): size() == old_size + 1]];
```

This would cover use cases:

- [api.establish.changedvalues](#),
- [api.establish.changedmembers](#),
- [api.establish.changedstate](#).

2.3.4. Different but compatible preconditions and postconditions in derived classes

Although we do not know yet how this would be handled in detail, it remains compatible with our model if overriding functions in derived classes have relaxed preconditions, and under some conditions more constrained postconditions. This would address:

- [cppapi.class.preconditions](#),
- [cppapi.class.postconditions](#).

2.4. Use-Cases Incompatible with Our Model

The contract support facility described in this paper only allows declaring predicates which indicate a presence of a bug in the program when they evaluate to `false` a certain well-defined moments. Plus smoe second-level annotations for controlling the tools. This framework cannot be used to declare or check properties of functions, such as runtime complexity, thread-safety or having strong exception safety.

The following use cases would never be handled by the proposed framework:

- [api.extend.exceptionsafety](#),
- [api.extend.threadsafety](#),
- [api.extend.atomicity](#),
- [api.extend.realtime](#),
- [api.extend.determinism](#),
- [api.extend.purity](#),
- [api.extend.sideeffects](#),
- [api.extend.complexity](#).

3. Conclusion

While the proposed MVP is conservative, we believe that even the MVP may contain a number of unresolved small issues that will require time to polish. With EWG attention almost entirely consumed by controversial bits of C++20 contracts, the uncontroversial bits may not have been given sufficient consideration for attention. If we are to ship any contract programming support in C++23 time frame, we have to be sure that we have a small enough scope to be accommodated by the WG21 process.

The biggest advantage of this MVP is that we provide a standard notation for expressing preconditions and postconditions. This notation will be consumed by many tools, which are not necessarily compilers. The positive effect on the community is not necessarily in what compilers can do with it, but also what other automated tools can get from it. This is far beyond the scope of the C++ Standard.

4. References

- [P2114r0] — Joshua Berne, Ryan McDougall, Andrzej Krzemiński, "Minimal Contract Use Cases" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2114r0.html>).
- [P1995r0] — Joshua Berne, Timur Doumler, Andrzej Krzemiński, Ryan McDougall, Herb Sutter, "Contracts — Use Cases" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html>).
- [P2185r0] — Caleb Sunstrum, "Contracts Use Case Categorization" (<https://isocpp.org/files/papers/P2185R0.html>).
- [N4820] — Richard smith, "Working Draft, Standard for Programming Language C++", <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/n4820.pdf>.
- [P0542r5] — G. Dos Reis, J. D. Garcia, J. Lakos, A. Meredith, N. Myers, B. Stroustrup, "Support for contract based programming in C++" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0542r5.html>).
- [p2076r0] — Ville Voutilainen, "Previous disagreements on Contracts" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2076r0.html>).
- [P1774r3] — Timur Doumler, "Portable assumptions" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p1774r3.pdf>).

- [P1332r0] — Joshua Berne, Nathan Burgers, Hyman Rosen, John Lakos, "Contract Checking in C++: A (long-term) Road Map" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1332r0.txt>).
- [P1517r0] — Ryan McDougall, "Contract Requirements for Iterative High-Assurance Systems" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1517r0.html>).
- [P2176r0] — Andrzej Krzemiński, "A different take on inexpressible conditions" (<https://isocpp.org/files/papers/P2176R0.html>).
- [P2064r0] — Herb Sutter, "Assumptions" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2064r0.pdf>).
- [P2038r0] — Andrzej Krzemiński, Ryan McDougall, "Proposed nomenclature for contract-related proposals" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2038r0.html>).
- [N4160] — Andrzej Krzemiński, "Value Constraints" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4160.html>).
- [P1344r1] — Nathan Myers, "Pre/Post vs. Enspects/Exsures" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1344r1.md>).
- [P1711r0] — Bjarne Stroustrup, "What to do about contracts?" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1711r0.pdf>).